

# Software Architecture and Design Specification

---

## Project: Student Record System

Team No. 11

### INDIVIDUAL CONTRIBUTIONS

| Team Member              | SRN           | Contribution                                                                                                                                                               |
|--------------------------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HIMANI PRABHU            | PES2UG24CS194 | <b>Sections 1, 2, 4.5, 4.6 &amp; 5</b> — Introduction, document overview, UX design, open issues & next steps, and appendices                                              |
| KOJAGORI<br>CHAKRABORTY  | PES2UG24CS228 | <b>Sections 3.3, 3.4, 4.1, 4.2 &amp; 4.3</b> — Component UML, component descriptions, design overview, sequence diagrams, and interface design                             |
| KULAKARNI<br>VIJAYEENDRA | PES2UG24CS244 | <b>Sections 3.1, 3.2, 3.5, 3.6 &amp; 3.7</b> — Architecture goals & constraints, stakeholders, architecture pattern, technology stack/data stores, and risks & mitigations |
| IBRAHIM KHALEEL          | PES2UG25CS809 | <b>Sections 3.8, 3.9 &amp; 4.4</b> — Requirements traceability, security architecture, and error handling, logging & monitoring                                            |

## 1. Introduction

### 1.1 Purpose

This Software Architecture and Design Specification (SADS) describes the design context for the Student Record System. The system is intended to manage student personal information, subjects, grades, attendance, academic reports, and role-based access to student records. This section establishes the purpose and context for the architecture and design work.

### 1.2 Scope

The Student Record System covers computerized management of student records through functions such as student management, course/subject management, grade management, attendance management, record search, reporting, user/access management, data management, record administration, and system exit.

The scope does not include external LMS integrations, financial management such as fees or scholarships, or examination scheduling beyond grade entry. The basic implementation is intended as a standalone C/C++ system and does not require network communication.

### 1.3 Audience

This document is intended for the project development team, testers/QA members, instructors or evaluators, system administrators, and future maintainers who need to understand how the Student Record System is organized and used.

### 1.4 Definitions

- Student Record System (SRS): The application used to manage student personal and academic records.
- RBAC: Role-Based Access Control, used to restrict functions according to user role.
- Administrator: User responsible for authorized user-account and system-level management.
- Faculty/Teacher: User permitted to perform academic activities such as entering grades and attendance.
- Authorized Staff: User permitted to perform assigned student-record management activities.
- Student: User who can view permitted information, including their own student data.
- GPA/CGPA: Academic performance measures calculated from recorded grades.
- DBMS: Database Management System used for persistent record storage.

## 2. Document Overview

### 2.1 How to use this document

This document provides the architecture and design information for the Student Record System. The Introduction establishes the system context and scope. The UX Design section describes the user-facing design principles derived from the Student Record System requirements. Open Issues & Next Steps identifies possible future enhancements or scope-expansion areas. The Appendices provide supporting definitions, references, and design tools.

### 2.2 Related Documents

- Student Record System – Software Requirements Specification (SRS), Team 11.
- Student Record System – Software Test Plan (STP) and Test Cases.
- Requirements Traceability Matrix (RTM) contained in the final SRS.

The final SRS is the primary source for system scope, functional requirements, non-functional requirements, security requirements, acceptance criteria, and requirement-to-test traceability.

## 3. Architecture

### 3.1 Goals & Constraints

#### Architecture Goals

The architecture of the Student Record System is designed to achieve the following goals:

1. **Maintainability**  
The system shall use modular components so that student, subject, grade, attendance, and user-management functions can be maintained independently.
2. **Data Integrity**  
The architecture shall ensure consistency and integrity of student records, including student information, subjects, grades, and attendance.
3. **Security**  
The architecture shall protect student personal and academic information through authentication and role-based access control.
4. **Reliability**  
The system shall support reliable storage and retrieval of student records without unintended data loss during normal operation.
5. **Performance**  
The architecture shall support normal system operations and provide student record and search results within the specified performance requirement.
6. **Usability and Accessibility**  
The system shall provide clear menus, understandable messages, simple navigation, and applicable accessibility support.
7. **Portability**  
The architecture shall allow the system to operate in the supported desktop environments specified by the project.

These goals are directly aligned with the SRS quality attributes, which specify performance, reliability, usability, portability, maintainability, security, and data integrity. SE\_SRS

#### Constraints

The architecture is subject to the following constraints:

- The system is implemented as a **standalone C/C++ application**.
- The basic implementation does **not require network communication**.
- A database management system/persistent storage mechanism is required for storing student records.

- Only authorized users shall access protected student information.
- Access to system functions shall be restricted according to the user's assigned role.
- The system must maintain consistency between students, subjects, grades, and attendance records.
- The system must operate within the available desktop/laptop hardware and software environment.
- No external third-party service integration is required.
- The system must comply with the security requirements specified in the SRS, including password hashing, session timeout, failed-login handling, and RBAC.

### 3.2 Stakeholders & Concerns

| Stakeholder           | Role                                                                                    | Main Architectural Concerns                                                   |
|-----------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Administrator         | Manages student records, subjects, grades, attendance and user accounts                 | Full but controlled access, data integrity, security, reliable administration |
| Faculty / Teacher     | Records and updates grades and attendance and retrieves student records                 | Correct academic data, authorization, usability and reliable updates          |
| Authorized Staff      | Searches, views, updates and generates student-related records according to permissions | Role-based access, data confidentiality, ease of record management            |
| Student               | Views permitted personal and academic information                                       | Privacy, restricted access, usability and correct information                 |
| Developers            | Design, implement and maintain the system                                               | Modularity, maintainability, clear interfaces and portability                 |
| QA / Test Engineers   | Verify system requirements and quality attributes                                       | Testability, traceability, reliability, security and predictable behavior     |
| System Administrators | Maintain the operating environment, database and user accounts where applicable         | Reliable deployment, data storage, access configuration and maintainability   |
| Project Evaluators    | Assess whether the system satisfies project requirements                                | Requirement compliance, completeness, quality and traceability                |

#### Key Stakeholder Concerns

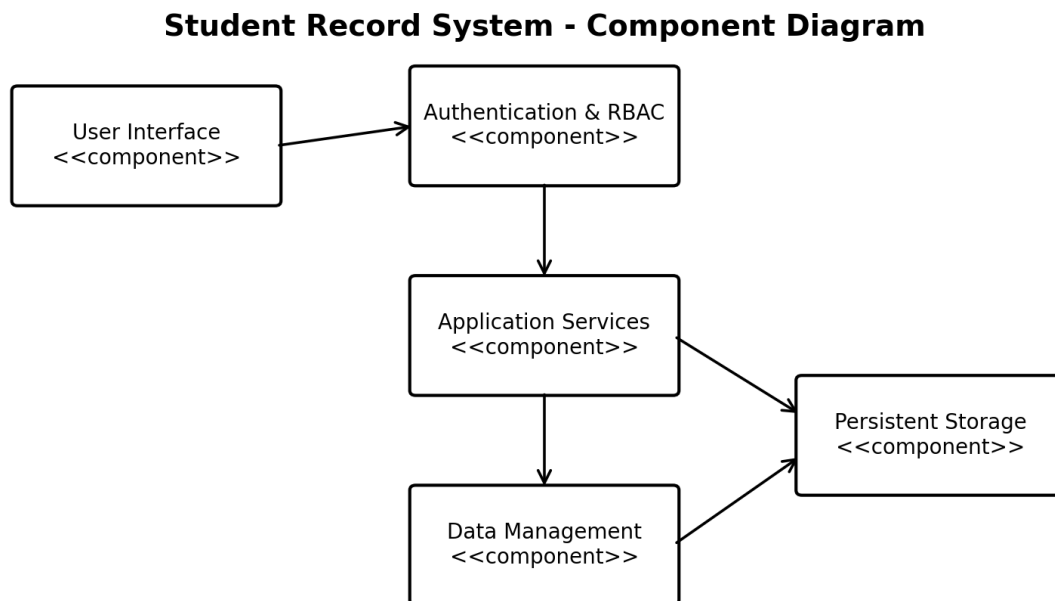
The architecture therefore needs to balance:

- **Security and confidentiality** of student information
- **Correctness and integrity** of academic records

- **Role-specific access**
- **Ease of use** for different user roles
- **Reliable persistent storage**
- **Maintainability** of individual system modules
- **Performance** during normal operation
- **Testability and traceability** against the SRS

### 3.3 Component (UML) Diagram

The component structure follows the layered architecture defined for the Student Record System. The User Interface communicates with authentication and application services, while the Data Management component controls validation and persistent record access.



*Figure 3.3: Component UML Diagram of the Student Record System*

### 3.4 Component Descriptions

**User Interface:** Provides the desktop interface for administrators, faculty/teachers, authorized staff, and students. It presents permitted menus, forms, search functions, reports, validation messages, and logout/exit options.

**Authentication & RBAC:** Authenticates users and determines their assigned role. It enforces role-based permissions so administrators, faculty/teachers, authorized staff, and students can access only permitted functions.

**Application Services:** Implements the main system operations, including student management, subject management, grade management, attendance management, search, reporting, and administrative functions.

**Data Management:** Validates required fields and record references, coordinates create/read/update operations, maintains relationships between students, subjects, grades, and attendance, and passes valid data to persistent storage.

**Persistent Storage:** Stores student records, subject records, grade records, attendance records, user-account information, and audit records so information remains available across application sessions.

## 3.5 Chosen Architecture Pattern and Rationale

### Chosen Architecture Pattern: Layered Architecture

The Student Record System follows a Layered Architecture that separates the system into distinct layers based on their responsibilities. The architecture consists of the presentation layer, application/logic layer, data management layer, and persistent storage layer.

The Presentation Layer provides the user interface through which administrators, faculty, authorized staff, and students interact with the system.

The Application/Logic Layer implements the main system functions, including student management, subject management, grade management, attendance management, search and reporting, authentication, and role-based access control.

The Data Management Layer handles data validation, record retrieval, updates, and communication with the persistent storage mechanism.

The Persistent Storage Layer stores student, subject, grade, attendance, and user-account information.

Authentication & RBAC acts as a cross-cutting application component that enforces authentication and authorization across protected system operations

### Rationale

Layered Architecture was selected because it provides clear separation of concerns between the user interface, application logic, and data management functions. This makes the system easier to develop, test, maintain, and modify.

The architecture is also suitable for the Student Record System because the SRS requires modular components for student, subject, grade, attendance, and user-management functions. It also supports the system's requirements for data integrity, security, maintainability, and reliable persistent storage.

The standalone nature of the application and the absence of a basic network communication requirement make a layered architecture appropriate for the scale and requirements of the system

## 3.6 Technology Stack & Data Stores

### Technology Stack

| Area                 | Selected Technology / Environment                      |
|----------------------|--------------------------------------------------------|
| Programming Language | C / C++                                                |
| Application Type     | Standalone desktop application                         |
| Operating System     | Windows or Linux                                       |
| User Interface       | Application interface suitable for desktop interaction |
| Persistent Storage   | Database Management System / persistent storage        |
| Input Devices        | Keyboard and optionally mouse                          |
| Output Device        | Display / monitor                                      |
| Network              | Not required for the basic implementation              |
| External Services    | No external third-party service integration            |

### Data Stores

The system requires persistent storage for the following categories of information:

| Data Store / Category | Purpose                                                            |
|-----------------------|--------------------------------------------------------------------|
| Student Records       | Student ID, name, contact and academic information                 |
| Subject Records       | Subject/course information and student-subject associations        |
| Grade Records         | Grades associated with students and subjects                       |
| Attendance Records    | Attendance information for students and subjects                   |
| User Account Records  | Authorized user accounts and role information                      |
| Audit Records         | CRUD activity and user/time information required for audit logging |

## 3.7 Risks & Mitigations

| Risk                                | Impact                                                                  | Mitigation                                                                                                                                  |
|-------------------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Data inconsistency between records  | Incorrect grades, attendance or academic information may be stored      | Validate required information and references before storing records; maintain consistency between students, subjects, grades and attendance |
| Unauthorized access                 | Sensitive student information may be exposed or modified                | Enforce authentication and role-based access control for all protected operations                                                           |
| Password/security weakness          | User credentials could be compromised                                   | Store passwords using secure hashing and never store plaintext passwords                                                                    |
| Database/persistence failure        | Student records may become unavailable or lost                          | Verify persistent storage configuration and maintain suitable backups/test-data copies                                                      |
| Invalid input                       | Incorrect or incomplete records may enter the system                    | Perform required-field and data-value validation before storing information                                                                 |
| Performance degradation             | Search and record operations may become slow with expected data volumes | Perform performance/scalability testing and verify the specified response-time requirement                                                  |
| Build or environment instability    | Testing and execution may be blocked                                    | Maintain a reproducible supported environment and resolve blocking build defects before testing                                             |
| Incorrect role configuration        | Users may receive permissions beyond their responsibilities             | Maintain separate accounts for all four roles and verify permissions through RBAC testing                                                   |
| Requirement/implementation mismatch | The implementation may fail to satisfy SRS requirements                 | Use the final SRS and RTM as the source of truth and maintain requirement-to-test traceability                                              |



## 3.8 Requirements Traceability

Requirements traceability ensures that every requirement defined in the Student Record System SRS is linked to one or more architecture/design elements and to the test cases that verify it. The RTM in the final SRS is the single source of truth.

**Forward traceability** maps each SRS requirement to its architectural component, design artifact, and test case. **Backward traceability** maps each test case and design element back to at least one SRS requirement. This ensures no requirement is left without a design or verification path, and no design element or test exists without justification.

### Requirement Groups and Architectural Mapping

| Requirement Range     | Area                    | Architectural Component(s)                                                | Design Artifact                                                | Test Cases                       |
|-----------------------|-------------------------|---------------------------------------------------------------------------|----------------------------------------------------------------|----------------------------------|
| SRS-F-001 – SRS-F-004 | Student Management      | User Interface, Application Services, Data Management, Persistent Storage | Component Diagram (Fig 3.3), API addStudent(student) (Sec 4.3) | TC-STD-01 – TC-STD-04            |
| SRS-F-005 – SRS-F-007 | Subject Management      | Application Services, Data Management                                     | Component Diagram, Data Store: Subject Records                 | TC-SUB-01 – TC-SUB-03            |
| SRS-F-008 – SRS-F-011 | Grade Management        | Application Services, Data Management                                     | Sequence Diagram Fig 4.2.2, API updateGradeAttendance(...)     | TC-GRD-01 – TC-GRD-04            |
| SRS-F-012 – SRS-F-014 | Attendance Management   | Application Services, Data Management                                     | Sequence Diagram Fig 4.2.2                                     | TC-ATT-01 – TC-ATT-03            |
| SRS-F-015 – SRS-F-017 | Search & Reporting      | User Interface, Application Services                                      | Component Diagram, UX Design Sec 4.5                           | TC-SRCH-01, TC-RPT-01, TC-RPT-02 |
| SRS-F-018 – SRS-F-020 | Authentication & Access | Authentication & RBAC                                                     | Sequence Diagram Fig 4.2.1, API                                | TC-AUTH-01 – TC-AUTH-03          |

| Requirement Range       | Area                                                                                                          | Architectural Component(s)                                 | Design Artifact                                | Test Cases                                                                    |
|-------------------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|------------------------------------------------|-------------------------------------------------------------------------------|
|                         |                                                                                                               |                                                            | authenticateUser(username, password)           |                                                                               |
| SRS-F-021 – SRS-F-023   | Data Management                                                                                               | Data Management, Persistent Storage                        | Component Diagram, Data Stores Table (Sec 3.6) | TC-DATA-01 – TC-DATA-03                                                       |
| SRS-F-024 – SRS-F-025   | Exit & Administration                                                                                         | User Interface, Application Services                       | Component Diagram                              | TC-EXIT-01, TC-ADMIN-01                                                       |
| SRS-NF-001 – SRS-NF-007 | Non-functional (performance, reliability, data protection, audit retention, accessibility, scalability, RBAC) | All layers                                                 | Architecture Goals Sec 3.1, Risks Sec 3.7      | TC-Perf-01, TC-REL-01, TC-Sec-01, TC-OPS-01, TC-UX-01, TC-Perf-02, TC-Auth-02 |
| SRS-SR-001 – SRS-SR-005 | Security                                                                                                      | Authentication & RBAC, Data Management, Persistent Storage | Security Architecture Sec 3.9                  | TC-Sec-01 – TC-Sec-05                                                         |

## 3.9 Security Architecture

### Security Design Principles

1. **Least Privilege** — every user receives only the permissions required for their role.
2. **Defense in Depth** — security is enforced at the UI, application, data-management, and storage layers, not at a single point.
3. **Fail Secure** — on authentication or authorization failure, access is denied by default.
4. **Separation of Duties** — administrative, academic, and viewing functions are separated by role.

5. **Auditability** — all security-relevant and CRUD actions are logged with user, time, and action information.

## Authentication

Users authenticate through the Authentication & RBAC component using `authenticateUser(username, password)`. Credentials are validated against stored user-account records. On success, the user's assigned role is returned and used to configure permitted UI functions. On failure, access is denied and the failure is logged (SRS-F-018).

## Password Storage

All user authentication uses secure password hashing (**bcrypt/Argon2**). Passwords are never stored in plaintext. Plaintext passwords are never written to storage, logs, or audit records (SRS-SR-002).

### RBAC:

| Role             | Permitted                             | Restricted                                       |
|------------------|---------------------------------------|--------------------------------------------------|
| Administrator    | Full system access                    | None beyond system-level                         |
| Faculty/Teacher  | Grades, attendance, permitted records | User-account mgmt, system admin                  |
| Authorized Staff | Search/view/update per permissions    | Grade mod outside scope, user-account mgmt       |
| Student          | View own permitted info               | Modify protected records, access others' records |

## Session Management

A **15-minute inactivity session timeout** is enforced for all users (SRS-SR-003). After timeout, the session is terminated and the user must re-authenticate. Logout or exit safely terminates the active session (SRS-F-024).

## Failed-Login Handling

Failed login attempts are **logged with a timestamp** (SRS-SR-004). After **five consecutive failed attempts**, the account is restricted and further access is blocked. Failed-login handling is integrated with audit logging.

## Data Protection

Unauthorized access to protected student information is rejected (SRS-SR-001). Sensitive information is not stored in plaintext where protection is required. An encryption audit must pass, and no plaintext PII is present in logs or database (SRS-NF-003). Data protection is enforced across the Data Management and Persistent Storage layers.

**Audit Logging**

CRUD operations are logged with user, time, and action information (SRS-NF-004). Audit logs are **retained for 5 years**. Audit records support security investigation and compliance verification.

**Security Control**

| Control                                 | Requirement            | Test Case |
|-----------------------------------------|------------------------|-----------|
| Unauthorized-access protection          | SRS-SR-001, SRS-NF-003 | TC-Sec-01 |
| Password hashing (bcrypt/Argon2)        | SRS-SR-002             | TC-Sec-02 |
| 15-minute inactivity timeout            | SRS-SR-003             | TC-Sec-03 |
| Failed-login logging and lock           | SRS-SR-004             | TC-Sec-04 |
| Role-specific restrictions              | SRS-SR-005             | TC-Sec-05 |
| CRUD audit logging and 5-year retention | SRS-NF-004             | TC-OPS-01 |

### 4.1 Design Overview

The Student Record System is designed as a layered standalone C/C++ application. The design separates presentation, application/logic, data management, and persistent storage responsibilities. The presentation layer handles user interaction; the application/logic layer coordinates authentication, RBAC and record-management operations; the data management layer validates and retrieves data; and the persistent storage layer maintains student, subject, grade, attendance, user-account, and audit information.

The design supports the SRS requirements for modularity, data integrity, security, reliability, performance, usability, accessibility, and portability. Interfaces between components are kept focused so that individual modules can be tested and maintained without changing unrelated modules.

### 4.2 UML Sequence Diagrams

The following sequence diagrams describe two project-specific flows: user authentication with role-based access and academic record entry/update by a faculty user.

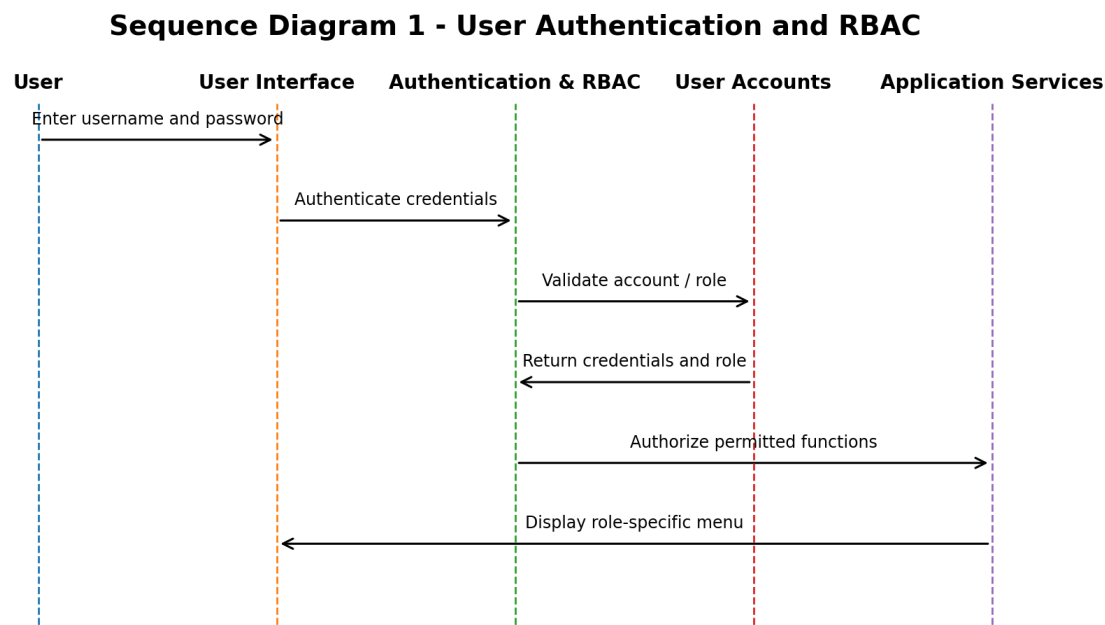


Figure 4.2.1: User Authentication and Role-Based Access Flow

## Sequence Diagram 2 - Enter Grade / Update Attendance

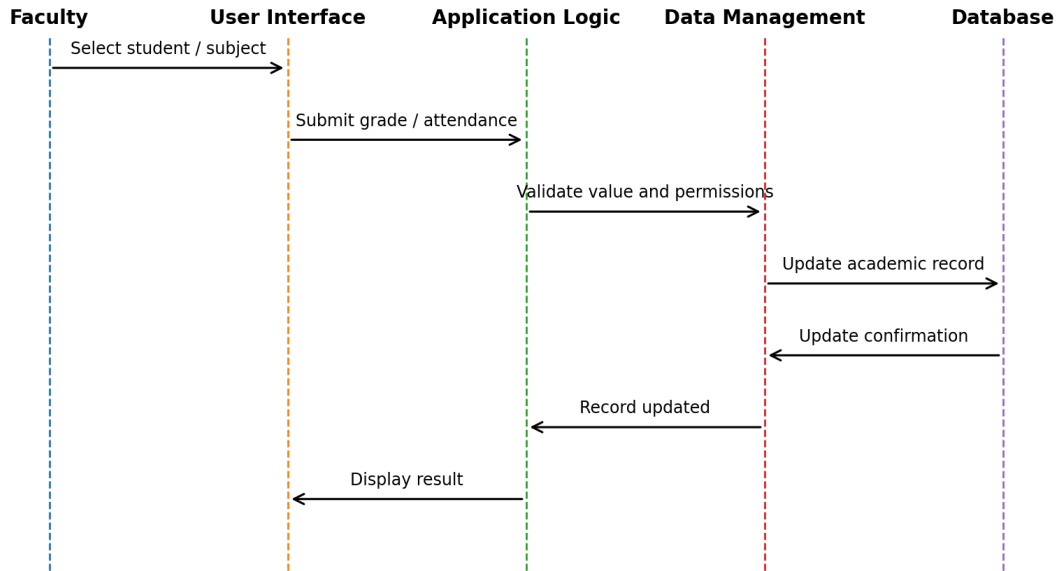


Figure 4.2.2: Grade and Attendance Update Flow

## 4.3 API Design

Because the basic system is a standalone C/C++ application and does not require network communication, the API design is expressed as internal component interfaces rather than HTTP endpoints.

The interfaces above are internal module-level interfaces. They support the layered design and keep validation, authorization, business logic, and storage responsibilities separated. Error conditions include invalid credentials, duplicate student IDs, missing required information, invalid grade or attendance values, unauthorized operations, and invalid record references.

| Component             | Interface                                                      | Input                                           | Output / Behaviour                                                            |
|-----------------------|----------------------------------------------------------------|-------------------------------------------------|-------------------------------------------------------------------------------|
| Authentication & RBAC | authenticateUser(username, password)                           | Username, password                              | Returns authentication result and assigned role; rejects invalid credentials. |
| Student Management    | addStudent(student)                                            | Student ID, name, contact details               | Validates required data and duplicate ID; stores the student record.          |
| Academic Records      | updateGradeAttendance(studentId, subjectId, grade, attendance) | Student ID, subject ID, grade, attendance value | Validates values and permissions, then updates the academic record.           |

## 4.4 Error Handling, Logging & Monitoring

1. **Fail safely** — errors must not corrupt stored records or expose protected data.
2. **Inform the user** — errors are reported through clear, understandable messages.
3. **Preserve data integrity** — invalid operations are rejected before storage.
4. **Log for diagnosis** — errors are recorded with sufficient context for investigation.

### Error Categories and Handling

| Error Category       | Example                                                | Handling                                                   | Related Requirement              |
|----------------------|--------------------------------------------------------|------------------------------------------------------------|----------------------------------|
| Authentication error | Invalid username/password                              | Reject access, log failure, increment failed-login counter | SRS-F-018, SRS-SR-004            |
| Authorization error  | Student attempts to edit grades                        | Block operation, display unauthorized message, log attempt | SRS-F-019, SRS-F-020, SRS-SR-005 |
| Validation error     | Missing required field, invalid grade/attendance value | Reject record, display validation message, do not store    | SRS-F-009, SRS-F-022             |
| Reference error      | Grade stored against non-existent student/subject      | Reject operation, display invalid-reference message        | SRS-F-023                        |
| Duplicate error      | Duplicate student ID                                   | Reject creation, display duplicate-ID message              | SRS-F-004                        |

| Error Category    | Example                                     | Handling                                     | Related Requirement   |
|-------------------|---------------------------------------------|----------------------------------------------|-----------------------|
| Persistence error | Database/storage unavailable                | Prevent data loss, display error, log event  | SRS-F-021, SRS-NF-002 |
| Session error     | Session timeout after 15 minutes inactivity | Terminate session, require re-authentication | SRS-SR-003            |

## Logging

- **Authentication events:** successful and failed logins, with timestamp and username.
- **Authorization events:** blocked unauthorized access attempts.
- **CRUD events:** create, read, update, delete actions with user, time, and action information (SRS-NF-004).
- **Validation events:** rejected records due to missing or invalid data.
- **System events:** persistence failures, session timeouts, and account lock events.
- Audit logs are retained for **5 years** (SRS-NF-004).

## Monitoring

- **Failed-login monitoring:** account is restricted after **5 consecutive failed attempts** (SRS-SR-004).
- **Session monitoring:** inactivity is tracked and sessions are terminated after **15 minutes** (SRS-SR-003).
- **Audit monitoring:** CRUD and security events are reviewed for anomalies.
- **Performance monitoring:** normal response times observed against the **95th percentile  $\leq$  3 seconds** requirement (SRS-NF-001).
- **Reliability monitoring:** normal operations monitored for unexpected termination or data loss (SRS-NF-002).

## Error Message Guidelines

- Messages are clear, consistent in terminology, and non-technical where possible.
- Messages state what went wrong and what action the user should take.
- Messages never reveal protected information or internal implementation details.
- Validation messages identify the specific field or value that is invalid.



## 4.5 UX Design

The Student Record System user experience should be clear, structured, and simple to navigate. The interface should present functions according to the user's role and should provide understandable prompts, validation messages, and error messages.

### 4.5.1 Navigation and Layout

- Use clear, structured menus for student, subject, grade, attendance, search, reporting, and administration functions.
- Keep navigation simple so users can move between permitted functions without unnecessary steps.
- Present relevant information in readable lists, forms, grade sheets, attendance summaries, and academic reports.
- Provide clear options for logout/exit and confirmation where an operation may affect stored records.

### 4.5.2 Input and Validation Experience

- Prompt users for required information before a record is saved.
- Display understandable messages when required data is missing or invalid.
- Reject invalid grades, attendance values, duplicate student IDs, and invalid record references according to the system requirements.
- Provide feedback after successful add, update, search, or report-generation operations.

### 4.5.3 Accessibility and Usability

- Support keyboard navigation and avoid requiring mouse-only interaction.
- Where applicable to the selected UI technology, support screen-reader use and high-contrast presentation.
- Follow WCAG 2.1 AA accessibility expectations where applicable.
- Use consistent terminology and simple messages so users can understand what action is required.

### 4.5.4 Role-Aware Experience

The interface should expose only the functions permitted for the authenticated role. Administrators have full system-level access; Faculty/Teachers manage permitted academic information such as grades and attendance; Authorized Staff perform permitted record-management activities; Students are restricted to permitted viewing of their own information. Students must not be provided with controls that allow modification of protected academic records.

## 4.6 Open Issues & Next Steps

The final SRS defines a focused standalone Student Record System. The following items are therefore treated as possible future enhancements or scope-expansion topics rather than current required features.

### 4.6.1 Future Enhancements

- Consider integration with external Learning Management Systems (LMS) if integration becomes a future project requirement.
- Consider a separate financial-management module for fees or scholarships if such functionality is later brought into scope.
- Consider examination-scheduling functionality beyond the current grade-entry scope.
- Consider a future network-based deployment if the project later requires multi-user or distributed access.

- Continue improving accessibility and usability based on testing feedback and the supported UI environment.

#### 4.6.2 Next Steps

1. Finalize the architecture and design sections for the remaining team members using the final SRS as the source of truth.
2. Ensure component, sequence, security, and traceability design artifacts remain consistent with the defined functional and non-functional requirements.
3. Validate the UX against the usability, accessibility, authentication, role-based access, validation, and data-integrity requirements.
4. Review open issues before implementation to decide which items remain outside the current release and which may be considered for a future version.

## 5. Appendices

### 5.1 Glossary

**SRS:** Software Requirements Specification.

**SAD/SADS:** Software Architecture and Design Specification.

**RBAC:** Role-Based Access Control.

**DBMS:** Database Management System.

**GPA:** Grade Point Average.

**CGPA:** Cumulative Grade Point Average.

**LMS:** Learning Management System.

**UI:** User Interface.

**WCAG:** Web Content Accessibility Guidelines.

### 5.2 References

- Student Record System – final Software Requirements Specification, Team 11.
- Student Record System – Software Test Plan and Test Cases.
- Requirements Traceability Matrix (RTM) in the final Student Record System SRS.
- WCAG 2.1 AA accessibility guidance, where applicable to the selected user-interface technology.

### 5.3 Tools

- C/C++ development environment and compiler, consistent with the SRS implementation approach.
- DBMS/data-storage tools for persistent student-record management.
- UML/design diagramming tool for architecture and design artifacts.
- Microsoft Word or equivalent document editor for maintaining the specification.

